AGENTIC AI ENGINEERING

From first model calls to safe agents,
multi-agent systems and reusable AI harnesses

5 DAYS	5 PROJECTS	BEGINNER TO ADVANCED

INCLUDED WITH THE COURSE

₹100 worth of AI API credits

Individually limited classroom access for guided model exercises

Designed for final-year engineering students with Python fundamentals and no prior agentic AI experience.

Project-based • Notebook-first • OpenRouter + open-source tooling • Mock fallbacks • No CrewAI or AutoGen

Build the system, not just a demo

This course introduces agentic AI one layer at a time. Students first use a model, then inspect structured output, tools and state. Knowledge, memory, safety and observability are added only when the project exposes a real limitation. The course ends by consolidating these ideas into a reusable AI harness.

MODEL	TOOL	AGENT	KNOWLEDGE	MEMORY	SAFETY	OBSERVABILITY	MULTI-AGENT	HARNESS

What makes the course different

Mechanism before framework

Students build and inspect the loop before using LangGraph. The subject is agentic system design, not memorising a framework.

Project-based progression

Five independent projects follow Build - Observe - Break - Improve. Six pivotal stub-and-test labs make students implement the central mechanisms.

Safe and measurable

Bounded steps, schema validation, permissions, approval, citations and golden-set evaluation are part of the core build.

Accessible classroom route

OpenRouter provides consistent model access. Mock paths keep every core lesson runnable during outages or debugging.

By the end, students can

- Build a bounded tool-using agent and explain who executes each action.
- Create a citation-producing RAG system and diagnose retrieval separately from generation.
- Add context management, persistent memory, policy and human approval.
- Compare single-agent and specialist multi-agent review systems using measurements.
- Build a mini harness that operates a guarded Website Maintenance Agent with persistent state and approval.

Syllabus: Foundations and knowledge

DAY 1 From model call to bounded agent

PROJECT	Smart Research Assistant
BUILD PATH	Model call → configuration → structured output → tool request → manual loop → LangGraph
CORE TOPICS	OpenRouter and mock routes; one direct OpenAI example; prompts and messages; Pydantic schemas; function calling; safe calculator; local search tool; tool-result messages; maximum-step termination.
TAKEAWAY	Students build an application in which the model may request tools, while Python validates and executes every action.

DAY 2 Knowledge, retrieval and visible state

PROJECT	Engineering Knowledge Assistant
BUILD PATH	Documents → chunks → keyword baseline → embeddings → retrieval → RAG → citations → evaluation
CORE TOPICS	Heading-aware chunking; lexical and semantic search; Sentence Transformers; Chroma; grounded generation; citation validation; abstention; retrieval and answer golden sets; retrieval as a tool; state inspection.
TAKEAWAY	Students build a small knowledge agent that answers from supplied engineering documents, cites evidence and abstains when evidence is insufficient.

Daily learning pattern

Each project begins with the smallest working version. Students inspect the actual messages, chunks, state or events; reproduce a supplied failure; add one layer; and complete a bounded code modification with an objective success condition.

Syllabus: Memory, safety and collaboration

DAY 3 Memory, planning and safe action

PROJECT	Safe Personal Task Agent
BUILD PATH	History → compaction → persistent memory → plan → side effects → policy → approval → events
CORE TOPICS	Conversation context and budgets; SQLite memory lifecycle; bounded plans; input, context, output, tool and execution guardrails; allow/approval/deny policy; injection defence; human-in-the-loop; safety cases and traces.
TAKEAWAY	Students build an agent where a mock or live model proposes an action, but application policy and explicit human approval control execution.

DAY 4 Measured multi-agent systems

PROJECT	Engineering Design Review Team
BUILD PATH	Seeded artifact → single reviewer → deterministic checks → specialists → supervisor → A/B
CORE TOPICS	Structured engineering findings; correctness, security and maintainability roles; deterministic AST checks; sequential and parallel fan-out/fan-in; supervisor validation and deduplication; recall, false positives, model calls, tokens, latency and cost.
TAKEAWAY	Students compare one general reviewer with bounded specialist reviewers and justify whether the added agents are worth the additional complexity.

A deliberate design choice

The single-agent system is allowed to win. Multi-agent architecture is introduced as a measurable decomposition technique - not as a default, a swarm, or a substitute for deterministic engineering tools.

Syllabus: The AI harness capstone

DAY 5	Reusable harness and operational automation
PROJECT	Mini AI Harness + Website Maintenance Agent
BUILD PATH	Runtime → registry → guardrails → MCP → scheduled check → approval → verified website update
CORE TOPICS	Mock/OpenRouter providers; registry and policy; events and checkpoints; MCP; cached or public update source; durable processed-item state; indirect-injection challenge; bounded live observation; persistent local website change; optional LLM judge.
TAKEAWAY	Students use their harness for a recurring real-file workflow while keeping model proposals, guardrails, approval, verification and scheduling as separate responsibilities.

Five independent portfolio projects

01	**Smart Research Assistant**	A bounded model-and-tool loop
02	**Engineering Knowledge Assistant**	RAG with citations and abstention
03	**Safe Personal Task Agent**	Memory, policy and human approval
04	**Engineering Design Review Team**	Measured single vs multi-agent review
05	**Mini AI Harness + Website Agent**	Guarded recurring workflow with persistent effect

No framework zoo

CrewAI and AutoGen are deliberately excluded. LangGraph is used after the manual loop is understood. LangSmith, Mem0 and MCP are shown as product or protocol exposures around transparent local mechanisms, not as definitions of the concepts themselves.

Practical information

Who should attend

Final-year engineering students, recent graduates and early-career developers who can read and modify basic Python but are new to agentic AI.

Prerequisites

Basic Python functions, lists, dictionaries and classes; package installation; notebooks; the basic idea of an HTTP API. Git is helpful but not required.

Delivery format

Five classroom days with theory embedded beside notebook code, deliberate failures, six pivotal exercise notebooks and one independently runnable project per day.

Student environment

Students work on their own computers. OpenRouter is the primary live route; Ollama is optional; deterministic fallbacks support low-spec systems and service outages.

₹100 WORTH OF AI API CREDITS INCLUDED

Each learner receives individually limited classroom API access for the guided AI exercises. Credits are intended for this curriculum and are complemented by mock fallbacks for debugging and outages.

Core technology exposure

Python • Jupyter • OpenRouter • GPT-OSS • Pydantic • LangGraph • Chroma • Sentence Transformers • SQLite • Mem0 • LangSmith • MCP

Optional and comparative: Ollama, direct OpenAI API, Langfuse, FastAPI and Docker.

What is intentionally outside the core

Agent swarms, CrewAI/AutoGen surveys, browser agents, model training, advanced Graph RAG, Kubernetes, remote production MCP authentication and enterprise deployment. These topics are deferred so students can understand the engineering foundations first.

LEARN THE LAYERS. BUILD THE SYSTEMS. ENGINEER THE HARNESS.